TMF 915 AI Management API

Bridging MLflow with Telecom Standards

OpenSlice — ETSI OSL

Agenda

1. **What is TMF 915?**
2. **What is MLflow?**
3. **How They Map Together**
4. **Architecture Overview**
5. **Lifecycle & State Machine**
6. **Automatic Model Discovery (Sync)**
7. **Docker-Based Deployment**
8. **Configuration & Deployment Guide**
9. **Demo & API Examples**

What is TMF 915?

The **TM Forum AI Management API (TMF 915)** is a standardized REST API for managing AI/ML assets in telecom environments.

Core Entities

Entity	Purpose
AiModelSpecification	Blueprint / template describing a model's capabilities
AiModel	Runtime instance of a deployed model
AiContract	SLA / agreement governing model usage
AiContractSpecification	Template for contracts
AiContractViolation	Records SLA breaches
Rule	Enforcement rules (thresholds, penalties)
Alarm	Operational alerts

TMF 915 — Key Relationships

```
AiModelSpecification (1) ──────< AiModel (many)
         |                            |
   specCharacteristic          serviceCharacteristic
   (model metadata,            (deployment metadata,
    capabilities)               endpoint, containerId)
         |                            |
AiContractSpecification ────── AiContract ────── AiContractViolation
                                      |
                                    Rule
```

- **One Spec → Many Models**: A single specification can spawn multiple deployed instances
- **Characteristics**: Key-value metadata pairs attached to both specs and models
- **Lifecycle**: Models follow a state machine (RESERVED → ACTIVE → TERMINATED)

What is MLflow?

MLflow is an open-source platform for managing the full ML lifecycle.

Key Concepts

Concept	Description
Experiment	A collection of training runs
Run	A single training execution with metrics, params, artifacts
Logged Model	A model artifact tracked within a run (MLflow 2.x+)
Model Registry	Classic versioned model catalog (name + version)
Artifacts	Stored model files, data, plots (S3/MinIO/local)

Two APIs

API	Endpoint	Used By

Why Bridge TMF 915 and MLflow?

MLflow Strengths

- Model training & tracking
- Experiment management
- Artifact versioning
- Metrics & parameters
- Python-native ecosystem

TMF 915 Strengths

- Standardized telecom API
- Lifecycle management
- Contract & SLA enforcement
- Event-driven notifications
- Multi-platform abstraction

The Bridge

> MLflow handles **model development** — TMF 915 handles **model operations**. Our integration automatically discovers MLflow models, converts them into TMF 915 entities, and deploys them as Docker containers with full lifecycle

Entity Mapping — Overview

```
┌──────────────────────────────┐                ┌──────────────────────────────┐
│          MLflow              │                │          TMF 915             │
│                              │                │                              │
│  Experiment                  │                │                              │
│    └─ Run                    │ ──────────▶    │  AiModelSpecification        │
│        └─ Logged Model       │   sync         │  (blueprint / template)      │
│           • model_id         │   service      │  • specCharacteristic[]      │
│           • artifact_uri     │                │  • metrics, params, tags     │
│           • metrics          │                │                              │
│           • params           │                │                              │
│           • model_type       │                │                              │
│                              │                │                              │
│  Docker Container            │ ◀──────────    │  AiModel                     │
│  (inference endpoint)        │   deploy       │  (runtime instance)          │
│                              │   service      │  • serviceCharacteristic[]   │
│                              │                │  • state, endpoint, port     │
└──────────────────────────────┘                └──────────────────────────────┘
```

Entity Mapping — AiModelSpecification

An **AiModelSpecification** is automatically created from each MLflow **Logged Model**.

MLflow Field	TMF 915 Field	Examp
`experiment.name`	`name`	`"fraud-detection"`
`loggedModel.info.model_id`	`version`	`"abc123def456"`
`loggedModel.info.model_type`	`specCharacteristic[modelType]`	`"sklearn"`
`loggedModel.info.artifact_uri`	`specCharacteristic[artifactUri]`	`"s3://mlflow/artifa`
`loggedModel.info.source_run_id`	`specCharacteristic[runId]`	`"run_789xyz"`
`loggedModel.data.metrics`	`specCharacteristic[metric_*]`	`accuracy=0.95`
`loggedModel.data.params`	`specCharacteristic[param_*]`	`n_estimators=100`
`loggedModel.info.tags`	`specCharacteristic[tags]`	`"production, v2"`
Tracking URI + experiment	`specCharacteristic[mlflowUrl]`	`http://mlflow:5000/`

Entity Mapping — AiModel

An **AiModel** is a running deployment instance, created when scheduling a model for serving.

Characteristic	Value	Set By
`platform`	`"mlflow"`	On creation
`deploymentTarget`	`"DOCKER"`	On creation
`mlflowModelId`	Logged model ID	On creation
`mlflowRunId`	Source run ID	On creation
`mlflowExperimentId`	Experiment ID	On creation
`endpoint`	`http://host:port/invocations`	After deploy
`containerId`	Docker container hash	After deploy
`containerName`	Human-readable name	After deploy

Architecture Overview

```
                    TMF 915 REST API  (/tmf-api/AiM/v4/)
                          │
                          ▼
                 ┌──────────────────┐
                 │ AiModelApi       │   ◀── Spring Boot Controller
                 │ Controller       │       @PreAuthorize(ROLE_USER)
                 └──────────────────┘
                          │
                          ▼
            ┌──────────────────────────┐
            │ AiModelLifecycleService  │   ◀── State validation
            │ (deployment/)            │       Default dates
            └──────────────────────────┘       CRUD orchestration
                          │
                          ▼
            ┌──────────────────────────┐
            │ DeploymentScheduler      │   ◀── ScheduledExecutorService
            │ (deployment/)            │       Startup recovery
            └──────────────────────────┘       Task tracking
                          │
                ┌─────────┴─────────┐
                ▼                   ▼
      ┌──────────────────┐   ┌──────────────────┐
      │ MlflowDeployer   │   │ Future Deployers │
      │ (PlatformDeployer)   │ ONNX, KServe ... │
      └──────────────────┘   └──────────────────┘
            │
      ┌──────────────────────────────────────────┐
      │          integrations/mlflow/            │
      │ MlflowModelService  → orchestration      │
      │ MlflowDeploymentService → Docker CLI     │
      │ MlflowClientService → MLflow REST API    │
      │ MlflowSyncService → periodic discovery   │
      │ MlflowSpecificationService → mapping     │
      └──────────────────────────────────────────┘
```

Platform Deployer — Strategy Pattern

The `PlatformDeployer` interface enables pluggable deployment backends.

```
public interface PlatformDeployer {
    boolean supports(AiModel model);   // Check platform characteristic
    void deploy(AiModel model);        // Build image + start container
    void undeploy(AiModel model);      // Stop container + cleanup
}
```

Current Implementation: `MlflowDeployer`

```
@Service
@ConditionalOnProperty(name = "mlflow.enabled", havingValue = "true")
public class MlflowDeployer implements PlatformDeployer {

    public boolean supports(AiModel model) {
        return "mlflow".equals(getCharacteristicValue(model, "platform"));
    }
}
```

Lifecycle State Machine

```
POST /aiModel                                          endDate reached
(state=RESERVED)                                            |
     |                                                      |
     ▼                startDate reached                     ▼
 RESERVED  ──────────────────────────────▶  ACTIVE  ──────▶  TERMINATED
 (scheduled)          deploy()              (serving)        (cleaned up)
     |                                          |
     |          PATCH state=inactive            |
     |                                          |
     ▼                                          ▼
 TERMINATED                              TERMINATED
 (cancelled)                             (user stop)
```

Scheduling Rules

Trigger	Action
`startDate` in the future	Schedule deploy for that time

Automatic Model Discovery — Sync Service

The `MlflowSyncService` periodically polls MLflow and creates missing specifications.

```
Every 60 seconds:

┌──────────────────────┐      GET /api/2.0/mlflow/
│  MlflowSync          │────   logged-models/search  ──▶┌──────────────────┐
│  Service             │                                │  MLflow          │
└──────────────────────┘                                │  Server          │
          │                                             └──────────────────┘
          │  For each logged model:
          ▼
┌──────────────────────────────────────────────────────┐
│  Does AiModelSpecification exist                     │
│  with name=experimentName                            │
│  and version=modelId ?                               │
└──────────────────────────────────────────────────────┘
      No │                 Yes
       ▼ │                  ▼
  Create new │            Skip
  specification
  with metrics,
```

Docker-Based Deployment Flow

When `MlflowDeployer.deploy()` is called:

```
Step 1: Build Docker Image
    ┌──────────────────────────────────────────────────────────┐
    │  mlflow models build-docker                              │
    │    --model-uri s3://mlflow/artifacts/.../model           │
    │    --name  fraud-detection-abc123                        │
    │    --env-manager local                                   │
    │                                                          │
    │  Sets: AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY,         │
    │        MLFLOW_S3_ENDPOINT_URL, DOCKER_HOST               │
    └──────────────────────────────────────────────────────────┘

Step 2: Deploy Container
    ┌──────────────────────────────────────────────────────────┐
    │  docker run -d                                           │
    │    --name fraud-detection-abc123                         │
    │    -p <scanned-port>:8080                                │
    │    fraud-detection-abc123                                │
    │                                                          │
    │  Port scan: tries 3000-7000 until one is free            │
    └──────────────────────────────────────────────────────────┘

Step 3: Update AiModel
    ┌──────────────────────────────────────────────────────────┐
    │  AiModel.state = ACTIVE                                  │
    │  + endpoint = http://host:port/invocations               │
```

Undeploy Flow

When the `endDate` is reached or the user patches `state=inactive`:

```
Step 1: Stop & Remove Container

    ┌──────────────────────────────────────────────────────────┐
    │  docker stop <containerId>                               │
    │  docker rm   <containerId>                               │
    └──────────────────────────────────────────────────────────┘

Step 2: Remove Docker Image

    ┌──────────────────────────────────────────────────────────┐
    │  docker rmi -f <imageName>                               │
    └──────────────────────────────────────────────────────────┘

Step 3: Update AiModel

    ┌──────────────────────────────────────────────────────────┐
    │  AiModel.state = TERMINATED                              │
    └──────────────────────────────────────────────────────────┘
```

Infrastructure Requirements

```
┌──────────────────────────────────────────────────────────────────────┐
│                        Deployment Infrastructure                     │
│                                                                      │
│  ┌──────────────┐   ┌──────────────┐   ┌──────────────┐   ┌──────────────────┐
│  │  MLflow      │   │  MinIO       │   │  Docker      │   │  MySQL /         │
│  │  Server      │   │  (S3)        │   │  Engine      │   │  MariaDB         │
│  │  :5000       │   │  :9000       │   │  :2375       │   │  :3306           │
│  └──────────────┘   └──────────────┘   └──────────────┘   └──────────────────┘
│       │                  │                  │                  │
│       └──────────────────┴──────────────────┴──────────────────┘
│                                  │
│                                  ▼
│                     ┌──────────────────────┐
│                     │  TMF 915 Service     │
│                     │  (Spring Boot)       │
│                     │  :13082              │
│                     └──────────────────────┘
│
│  Also required on the TMF 915 host:
│  • Java 17+              • Python 3.10+ (venv with mlflow + boto3)
│  • Maven 3.9+            • Docker CLI 29+
└──────────────────────────────────────────────────────────────────────┘
```

Configuration — application.yml

```yaml
# MLflow Integration
mlflow:
  enabled: true
  tracking-uri: "http://mlflow-server:5000"
  s3-endpoint-url: "http://minio:9000"
  s3-access-key: "minioadmin"
  s3-secret-key: "minioadmin"

  sync:
    enabled: true            # Auto-discover models
    interval-ms: 60000       # Poll every 60s

  docker:
    host: "tcp://docker-host:2375"
    container-port: 8080     # Internal container port
    host-port-start: 3000    # Port scan range start
    host-port-end: 7000      # Port scan range end
    env-manager: "local"     # local | virtualenv | uv

# AI Model defaults
aimodel:
  deployment:
    default-platform: mlflow
```

Step-by-Step Deployment Guide

1. Prerequisites

```
# Java & Maven
java -version    # 17+
mvn -version     # 3.9+

# Python environment
python3 -m venv .venv && source .venv/bin/activate
pip install mlflow boto3

# Docker
docker --version  # 29+
```

2. Infrastructure

Service	URL	Purpose
MLflow Server	`http://<host>:5000`	Model tracking & registry

Step-by-Step Deployment Guide (cont.)

3. Configure & Build

```
# Edit src/main/resources/application.yml with your infrastructure URLs

# Build
mvn clean package -DskipTests

# Or build + test (requires all infrastructure running)
PATH="$(pwd)/.venv/bin:$PATH" mvn clean test -Dspring.profiles.active=integration
```

4. Run

```
# Ensure mlflow CLI is on PATH
export PATH="$(pwd)/.venv/bin:$PATH"

# Start the service
java -jar target/controllers-tmf915-*.jar
```

API Usage — Create & Deploy a Model

1. Specifications are auto-discovered (or create manually)

```
curl -X POST http://localhost:13082/tmf-api/AiM/v4/aiModelSpecification \
  -H "Content-Type: application/json" \
  -d '{
    "name": "Fraud Detection Model",
    "version": "1.0",
    "description": "XGBoost fraud classifier",
    "lifecycleStatus": "Active"
  }'
```

2. Create an AiModel (RESERVED → auto-deploys at startDate)

```
curl -X POST http://localhost:13082/tmf-api/AiM/v4/aiModel \
  -H "Content-Type: application/json" \
  -d '{
    "name": "fraud-detector-prod",
    "state": "reserved",
```

API Usage — Monitor & Undeploy

3. Check deployment status

```
curl http://localhost:13082/tmf-api/AiM/v4/aiModel/<model-id>

# Response includes:
#   "state": "active",
#   "serviceCharacteristic": [
#     { "name": "endpoint", "value": "http://host:3042/invocations" },
#     { "name": "containerId", "value": "a1b2c3d4..." }
#   ]
```

4. Call the inference endpoint

```
curl -X POST http://<docker-host>:3042/invocations \
  -H "Content-Type: application/json" \
  -d '{"inputs": [[1.0, 2.0, 3.0, 4.0]]}'
```

End-to-End Flow Summary

```
Data Scientist                    TMF 915 Service                    Docker Host
      │                                │                                  │
      │  Train model in MLflow         │                                  │
      │ ──────────────────────────▶    │                                  │
      │                                │                                  │
      │                                │  Sync discovers new model        │
      │                ┌───────────────┤  every 60s                       │
      │                │ MlflowSyncService │                              │
      │                └───────────────┤                                  │
      │                                │                                  │
      │                          Creates AiModelSpecification            │
      │                                │                                  │
Operator                               │                                  │
      │  POST /aiModel                 │                                  │
      │  (state=reserved)              │                                  │
      │ ──────────────────────────▶    │                                  │
      │                                │  mlflow models build-docker      │
      │                                │ ─────────────────────────────▶   │
      │                                │                                  │
      │                                │  docker run -d -p port:8080      │
      │                                │ ─────────────────────────────▶   │
      │                                │                                  │
      │                          AiModel state = ACTIVE                  │
      │  ◀──────────────────────       │                                  │
      │  { endpoint: "http://..." }    │                                  │
      │                                │                                  │
Consumer                               │                                  │
      │  POST /invocations             │                                  │
      │ ──────────────────────────────────────────────────────────────▶  │
      │  ◀──────────────────────────────────────────────────────────────│
      │  { predictions: [...] }        │                                  │
```

Summary

Capability	How It Works
Model Discovery	Automatic sync from MLflow every 60s
Standardized API	Full TMF 915 v4.0.0 compliance
Lifecycle Management	RESERVED → ACTIVE → TERMINATED with scheduling
Docker Deployment	`mlflow models build-docker` + `docker run`
Multi-Platform	Pluggable `PlatformDeployer` interface
Crash Recovery	Pending deploys/undeploys re-scheduled on startup
S3 Artifacts	MinIO/S3 integration for model artifact storage
Extensible	Add new platforms without modifying core code

Thank You

Repository: `org.etsi.osl/controllers-tmf915`

API Docs: `http://localhost:13082/tmf-api/swagger-ui.html`

TMF 915 Specification: TM Forum AI Management API v4.0.0

MLflow: https://mlflow.org